

Assignment #5: cs201 Mock Exam寒露第三天

Updated 1913 GMT+8 Oct 10, 2025

2025 fall, Compiled by 同学的姓名、院系

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E29952: 咒语序列

Stack, <http://cs101.openjudge.cn/practice/29952/>

思路：

遇到（就入栈，遇到）就出栈，如果栈空了就说明序列断了

代码：

```

import sys
def solve():
    s=sys.stdin.read().strip()
    if not s:
        print(0)
        return
    stack=[-1]
    max_len=0
    for i, char in enumerate(s):
        if char=='(':
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)
            else:
                len_n=i-stack[-1]
                if len_n>max_len:
                    max_len=len_n
    print(max_len)
if __name__=='__main__':
    solve()

```

代码运行截图 （至少包含有"Accepted"）

M01328: Radar Installation

greedy, <http://cs101.openjudge.cn/practice/01328/>

思路：

将二维问题转化为一维贪心问题

代码：

```

import sys
import math
def solve():
    input_data=sys.stdin.read().split()
    if not input_data:
        return
    iterator=iter(input_data)
    case_num=1
    while True:
        try:
            n=int(next(iterator))
            d=int(next(iterator))
            if n==0 and d==0:
                break
            intervals=[]
            possibility= True
            for _ in range(n):
                x=int(next(iterator))
                y=int(next(iterator))
                if y>d:
                    possibility=False
                elif possibility:
                    delta=math.sqrt(d*d-y*y)
                    intervals.append((x-delta,x+delta))
            print(f"Case {case_num}: ",end="")
            if not possibility:
                print("-1")
            else:
                intervals.sort(key=lambda x: x[1])
                count=0
                last_radar_pos=-float('inf')
                for start, end in intervals:
                    if start>last_radar_pos:
                        last_radar_pos=end
                        count+=1
                print(count)
            case_num+=1
        except StopIteration:
            break
if __name__ == "__main__":
    solve()

```

代码运行截图（至少包含有"Accepted"）

#51508624提交状态

状态: Accepted

M02754: 八皇后

dfs, <http://cs101.openjudge.cn/practice/02754/>

思路:

通过递归与回溯，求出所有可能的解，按要求输出即可

代码:

```

solution=[]
def solve_queens(row,current_cols):
    if row==8:
        solution_str="".join(str(col+1)for col in current_cols)
        solution.append(solution_str)
        return
    for col in range(8):
        is_safe=True
        for r in range(row):
            c = current_cols[r]
            if col==c or abs(row-r)==abs(col-c):
                is_safe=False
                break
        if is_safe:
            current_cols[row]=col
            solve_queens(row+1,current_cols)
solve_queens(0,[0]*8)
solution.sort()
try:
    line=input()
    if line:
        n=int(line)
        for _ in range(n):
            b= int(input())
            print(solution[b-1])
except EOFError:
    pass

```

代码运行截图（至少包含有"Accepted"）

#51511551提交状态

状态: Accepted

M25570: 洋葱

matrices, <http://cs101.openjudge.cn/practice/25570/>

思路:

构造二维数组，回型遍历

代码:

```
import sys
def solve():
    n=int(input())
    matrix=[]
    for _ in range(n):
        row_data=sys.stdin.readline().split()
        row=list(map(int,row_data))
        matrix.append(row)
    max_sum=-float('inf')
    mat_lays=(n+1)//2
    for i in range(mat_lays):
        current_sum=0
        start=i
        end=n-1-i
        if start==end:
            current_sum+=matrix[start][start]
        else:
            for c in range (start, end+1):
                current_sum+=matrix[start][c]
            for c in range (start, end+1):
                current_sum += matrix[end][c]
            for r in range (start+1,end):
                current_sum+=matrix[r][start]
            for r in range (start+1,end):
                current_sum+=matrix[r][end]
        if current_sum>max_sum:
            max_sum=current_sum
    print(max_sum)
if __name__=='__main__':
    solve()
```

代码运行截图（至少包含有"Accepted"）

#51512814提交状态

状态: Accepted

M29954: 逃离紫罗兰监狱

bfs, <http://cs101.openjudge.cn/practice/29954/>

思路：

核心思路

题目要求“最少行动次数”。在网格图中，每一步移动的代价（时间）都是 1。BFS 能保证第一次到达终点时，所经过的步数一定是最少的。

普通的 BFS 只需要记录坐标 (r, c) 。但这道题有一个限制资源：法力值（即还能穿过多少次障碍物）。所以我们的状态需要包含三个变量： (r, c, k_used) ，分别代表：当前行、当前列、已消耗的闪现次数。

代码

```

import collections
def solve():
    try:
        line1=input().split()
        if not line1:
            return
        R, C, K=map(int,line1)
        grid=[input() for _ in range(R)]
    except EOFError:
        return
    start_pos=None
    for r in range(R):
        for c in range(C):
            if grid[r][c]=='S':
                start_pos=(r,c)
                break
    queue=collections.deque([(start_pos[0],start_pos[1],0,0)])
    min_k_map=[[float('inf')]*C for _ in range(R)]
    directions=[(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r,c,k_used,steps=queue.popleft()
        if grid[r][c]=='E':
            print(steps)
            return
        for dr,dc in directions:
            nr,nc=r+dr,c+dc
            if 0<=nr<R and 0<=nc<C:
                next_k=k_used
                if grid[nr][nc]=='#':
                    next_k+=1
                if next_k<=K and next_k<min_k_map[nr][nc]:
                    min_k_map[nr][nc]=next_k
                    queue.append((nr,nc,next_k,steps+1))
    print("-1")
if __name__=='__main__':
    solve()

```

(至少包含有"Accepted")

#51515545提交状态

状态: Accepted

T27256: 当前队列中位数

backtracking, <http://cs101.openjudge.cn/practice/27256/>

思路:

这个问题结合了队列 (FIFO) 的特性和有序集合的特性:

队列特性: add 在末尾加, del 在队头删。我们需要一个真正的 deque 来记录元素的插入顺序, 以便知道 del 时删的是哪个数。

中位数特性: 需要知道当前所有数排序后的中间位置。

最佳解决方案: 离散化 + 树状数组 (Fenwick Tree) 由于我们需要频繁地“插入数值”、“删除数值”和“查找第 K 小的数”, 树状数组 (BIT) 是一个非常高效的数据结构。

代码

```

import sys
from collections import deque
class FenwickTree:
    def __init__(self, size):
        self.tree = [0] * (size + 1)
        self.bit_len = size.bit_length()
        self.size = size

    def update(self, i, delta):
        while i <= self.size:
            self.tree[i] += delta
            i += i & (-i)

    def find_kth(self, k):
        idx = 0
        current_count = 0
        for i in range(self.bit_len - 1, -1, -1):
            next_idx = idx + (1 << i)
            if next_idx <= self.size and current_count + self.tree[next_idx] < k:
                idx = next_idx
                current_count += self.tree[idx]
        return idx + 1

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)
    try:
        n = int(next(iterator))
    except StopIteration:
        return

    ops = []
    all_numbers = set()
    for _ in range(n):
        cmd = next(iterator)
        if cmd == 'add':
            val = int(next(iterator))
            ops.append(('add', val))
            all_numbers.add(val)
        elif cmd == 'del':

```

```

ops.append(('del', None))
elif cmd == 'query':
    ops.append(('query', None))
sorted_nums = sorted(list(all_numbers))
val_map = {val: i + 1 for i, val in enumerate(sorted_nums)}
rank_map = {i + 1: val for i, val in enumerate(sorted_nums)}
max_rank = len(sorted_nums)

ft = FenwickTree(max_rank)
q = deque()
current_size = 0
output = []

for cmd, val in ops:
    if cmd == 'add':
        q.append(val)
        rank = val_map[val]
        ft.update(rank, 1)
        current_size += 1

    elif cmd == 'del':
        removed_val = q.popleft()
        rank = val_map[removed_val]
        ft.update(rank, -1)
        current_size -= 1

    elif cmd == 'query':
        if current_size % 2 == 1:
            k = (current_size + 1) // 2
            rank_k = ft.find_kth(k)
            median = rank_map[rank_k]
            output.append(str(median))
        else:
            k1 = current_size // 2
            k2 = k1 + 1

            rank1 = ft.find_kth(k1)
            rank2 = ft.find_kth(k2)

            val1 = rank_map[rank1]
            val2 = rank_map[rank2]
            avg = (val1 + val2) / 2
            if avg.is_integer():

```

```
        output.append(str(int(avg)))
    else:
        output.append(str(avg))

sys.stdout.write('\n'.join(output) + '\n')

if __name__ == '__main__':
    solve()
```

(至少包含有"Accepted")

#51517227提交状态

状态: Accepted

源代码

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。